

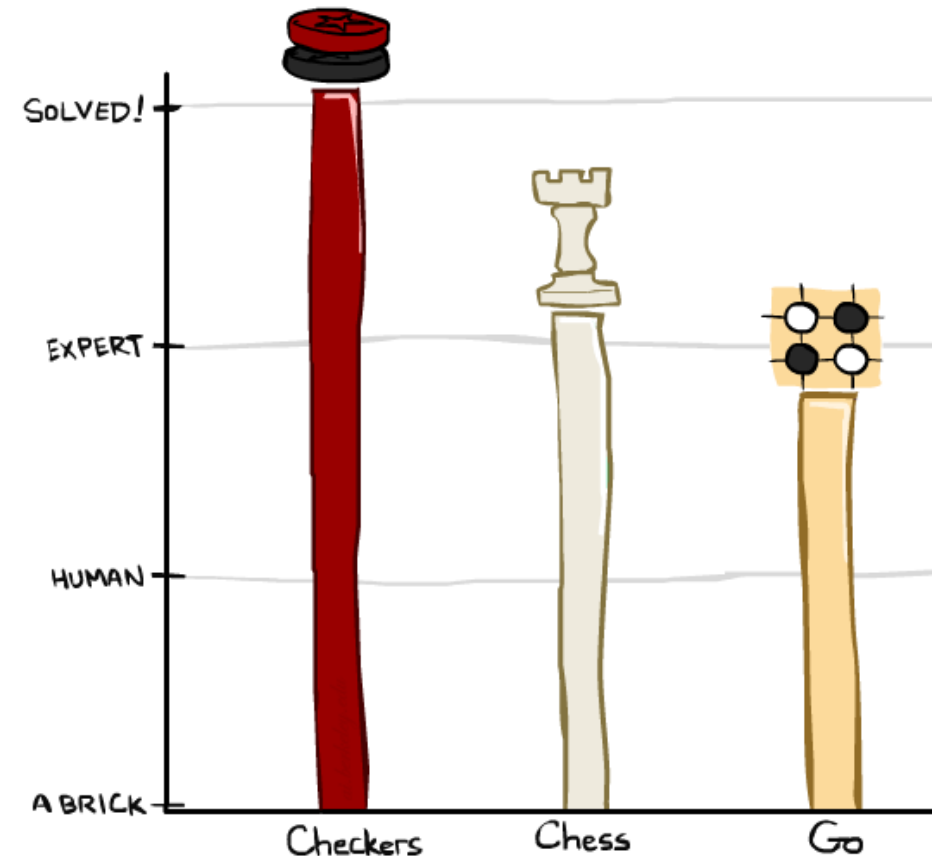
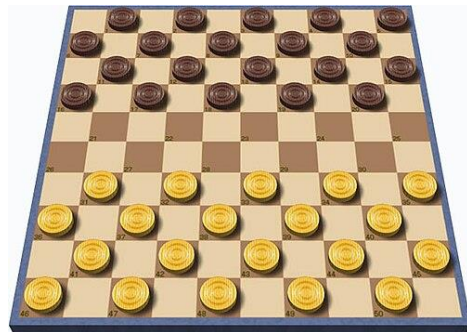
Adversarial Search and Games

Qian Wang

School of Biomedical Engineering
ShanghaiTech University

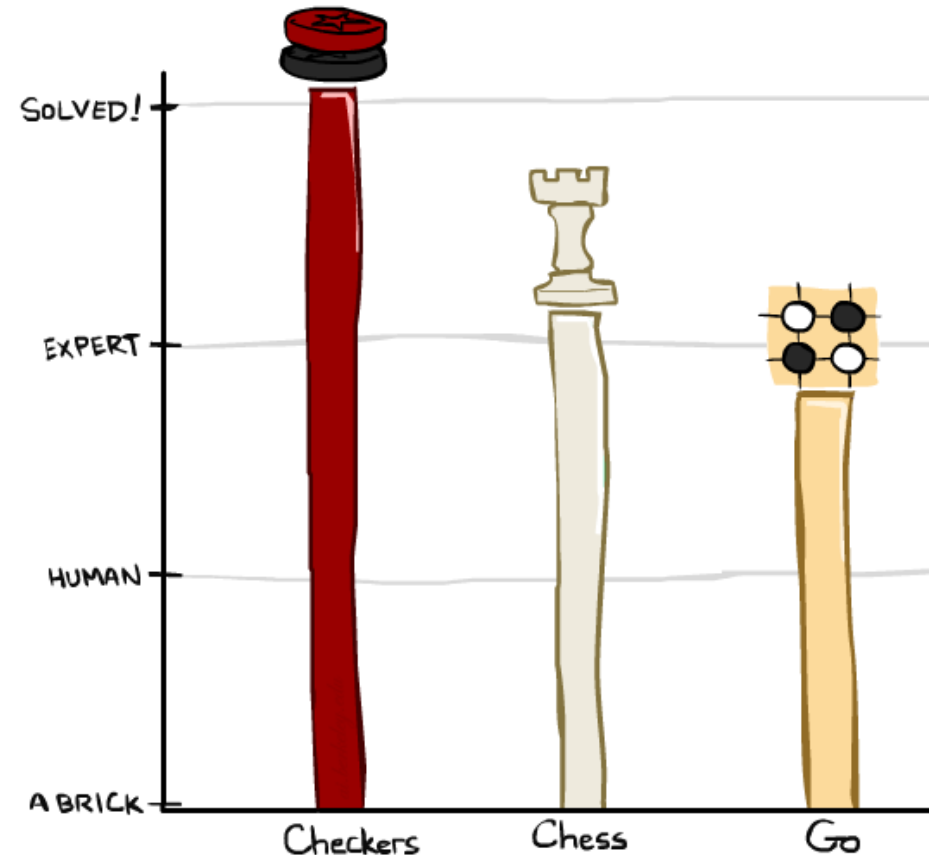
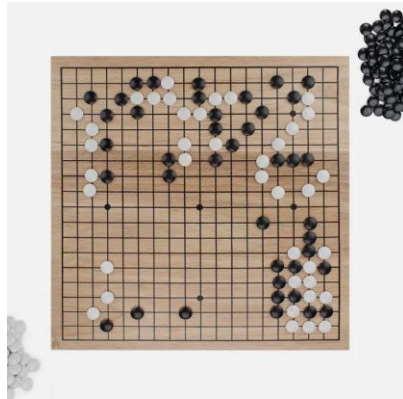
History of AI in Games

- Checkers
 - 1950: First computer program plays checkers;
 - 1994: Chinook becomes the first computer world champion;
 - 2007: Checkers is fully solved — perfect play from any position is known.



History of AI in Games

- Chess
 - 1997: Deep Blue defeats world champion Garry Kasparov in a six-game match.
- Go
 - 2016: AlphaGo defeats world champion Lee Sedol, marking a breakthrough in AI's ability to handle vast, intuitive search spaces



AlphaGo

- AlphaGo, a computer program that combines advanced search tree with deep neural networks.

Making history

AlphaGo is the first computer program to defeat a professional human Go player, the first to defeat a Go world champion, and is arguably the strongest Go player in history.

<https://www.deepmind.com/research/highlighted-research/alphago>



<https://www.bilibili.com/video/BV1gE411Y7F5/>

Why Study Games?

- Games are a classic benchmark of intelligence — they require planning, strategy, and adaptation.
- They serve as models for real-world interactions, including:
 - Competitive scenarios (e.g., military strategy, auctions);
 - Cooperative settings (e.g., negotiation, team coordination).
- Games are easy to formalize, making them ideal for algorithmic analysis:
 - Are outcomes deterministic or stochastic?
 - Are there one, two, or multiple players?
 - Is the game zero-sum, where one agent's gain is another's loss?

Types of Games

- General Games

- Each agent has its **own** utility function — independent preferences over outcomes;
- Interactions may involve cooperation, indifference, or competition;
- Captures a wide range of strategic behaviors.

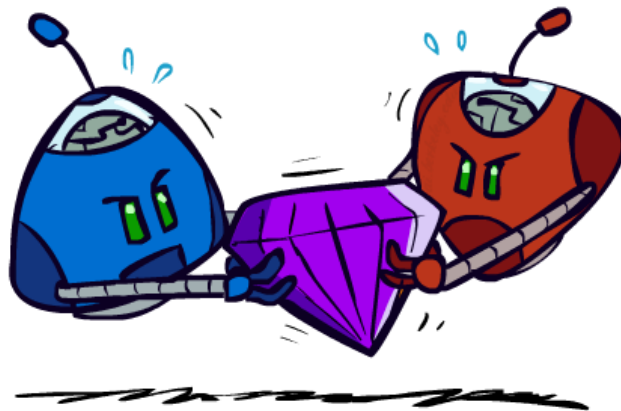
- Zero-Sum Games

- Agents have **opposing utilities**: one agent's gain is the other's loss;
- Can be modeled with a single value: **one agent maximizes, the other minimizes**;
- Represents pure **adversarial** competition.



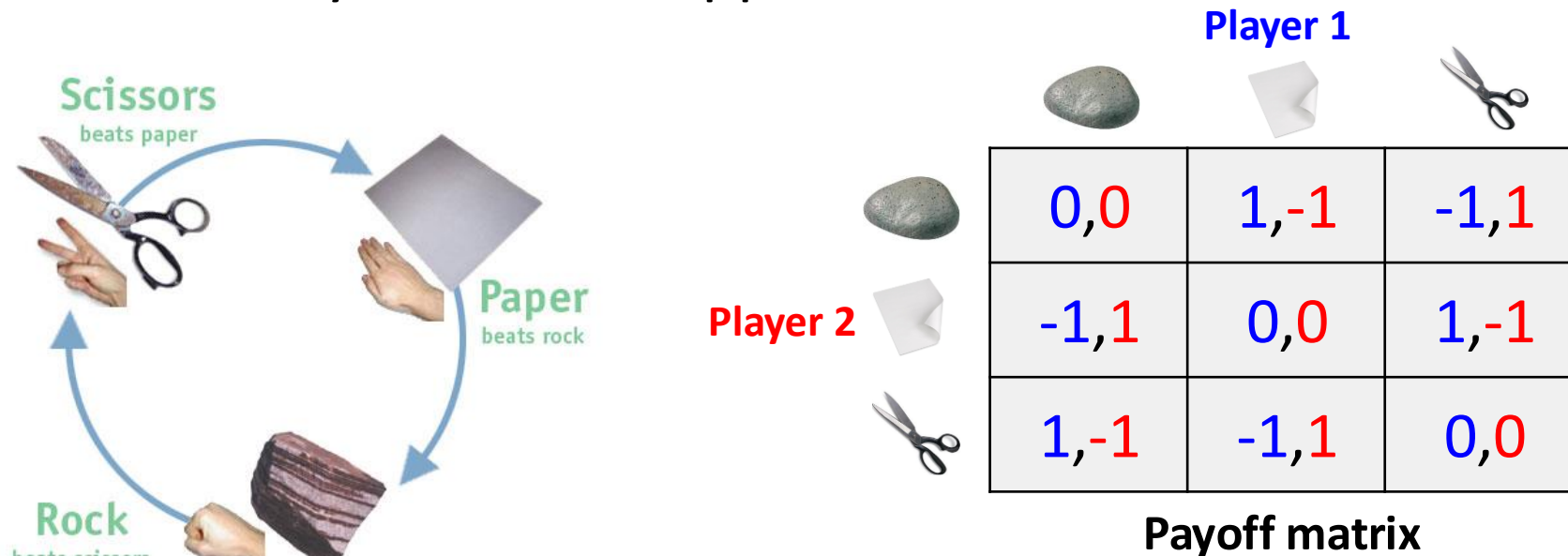
Alternating Two-player Zero-sum Games

- Players take turns, making moves in a discrete sequence.
- The sum of both players' utilities remains **constant** — typically zero, reflecting a zero-sum structure.
- Each terminal state (game outcome) assigns a **utility** value to each player.
 - Example: '1' for win, '0' or '-1' for loss.



Simultaneous Single-move Games

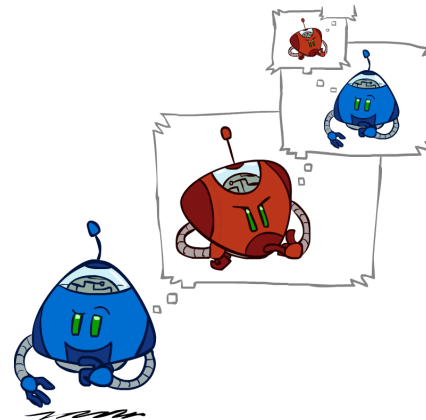
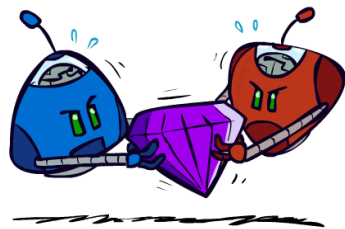
- Players must choose their actions at the same time, without knowing what the others will do.
- This setup introduces a form of partial observability — each agent must act under uncertainty about the opponent's move.



(Player 1's utility is listed blue, Player 2 in red)

Games vs. Single-agent Search

- In adversarial settings, we don't know how the opponent will act.
- Therefore, the solution is not a fixed sequence of actions from start to goal.
- Instead, we need a **strategy** or **policy**:
 - A **mapping** from each game state to the best possible move in that state;
 - Allows **dynamic response** to the opponent's choices.



Games vs. Single-agent Search

- **Efficiency** is critical to playing well.
 - The branching factor, search depth, and number of terminal states are huge.
 - In chess, **branching factor** ≈ 35 and **depth** ≈ 100 , giving a search tree of 10^{154} nodes.
 - Number of states in the observable universe $\approx 10^{80}$.
- The time to make a move is limited, which rules out exhaustively searching all the way to the end of the game.

Tic-Tac-Toe Game Tree



MAX (X)



MIN (O)



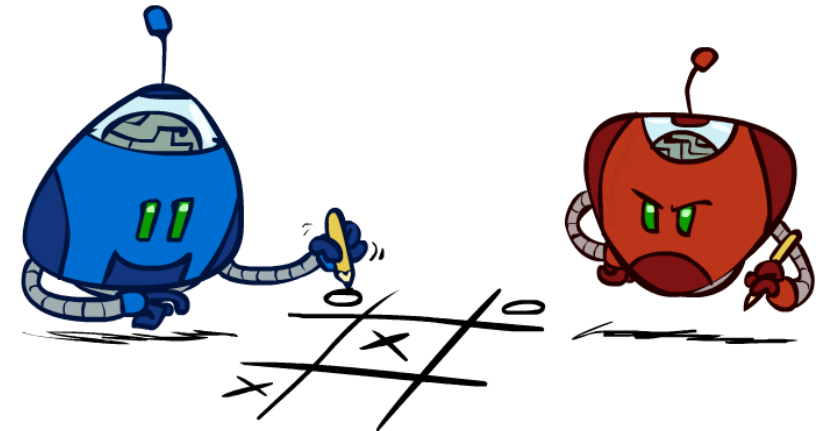
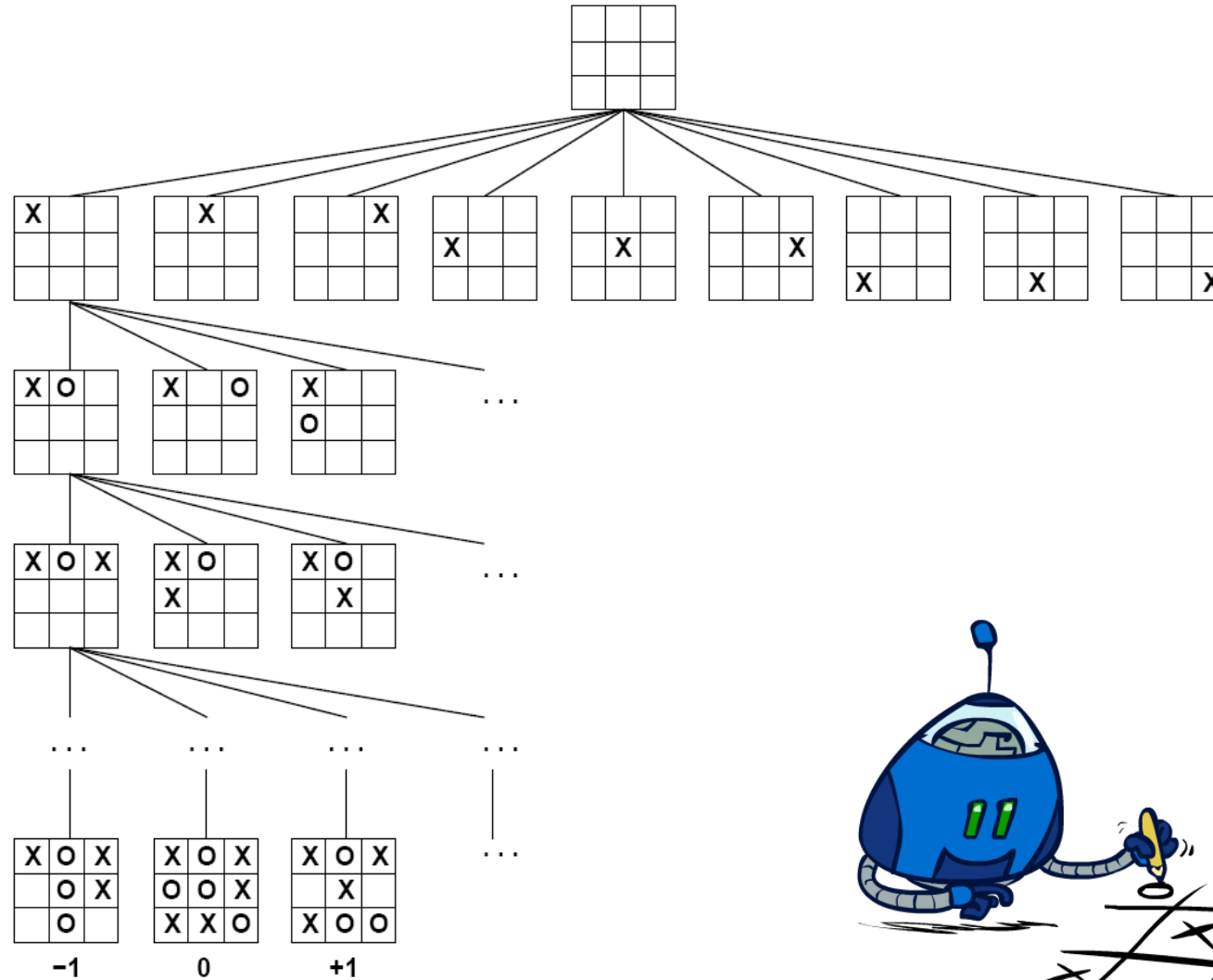
MAX (X)



MIN (O)

TERMINAL

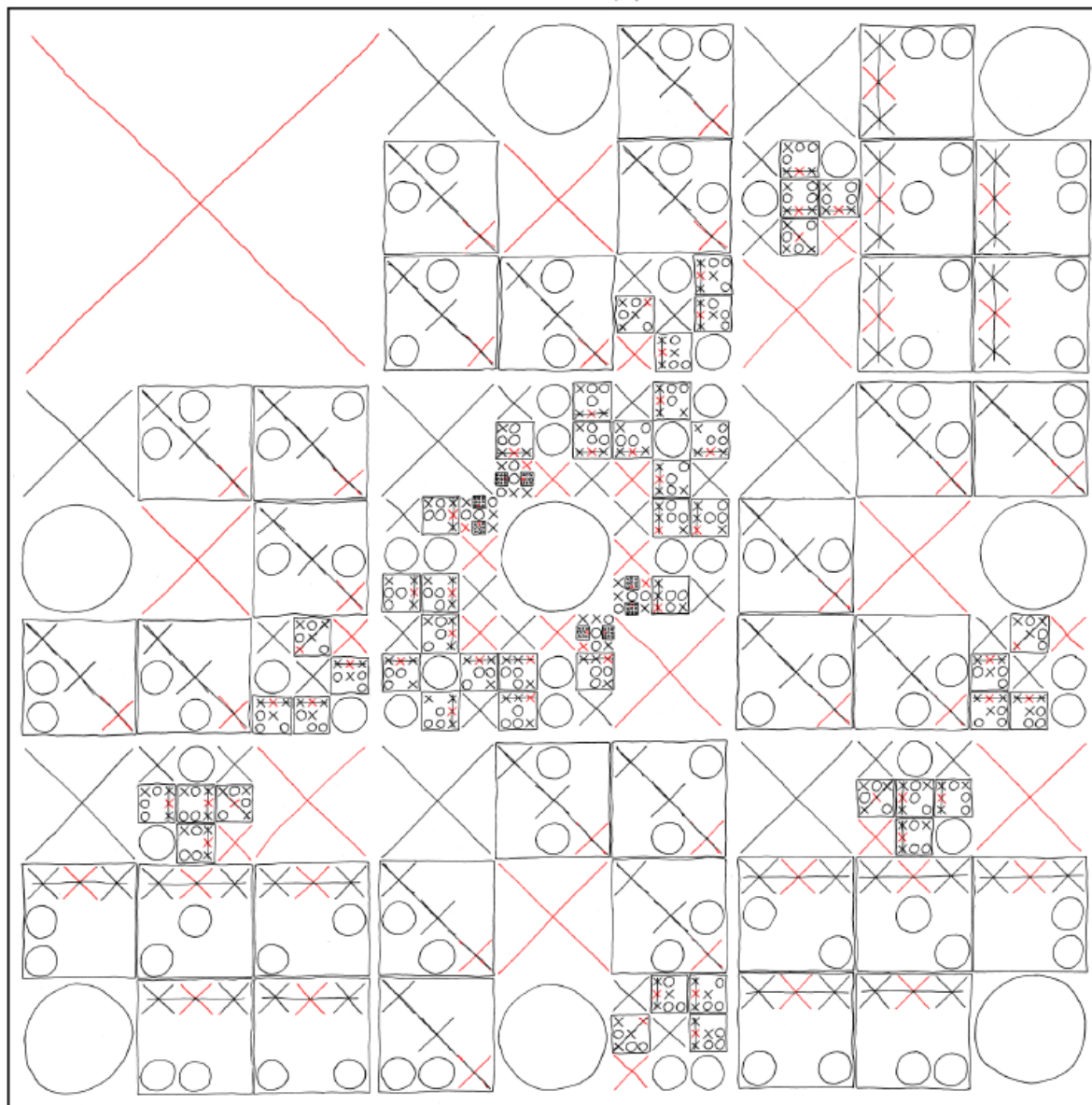
Utility



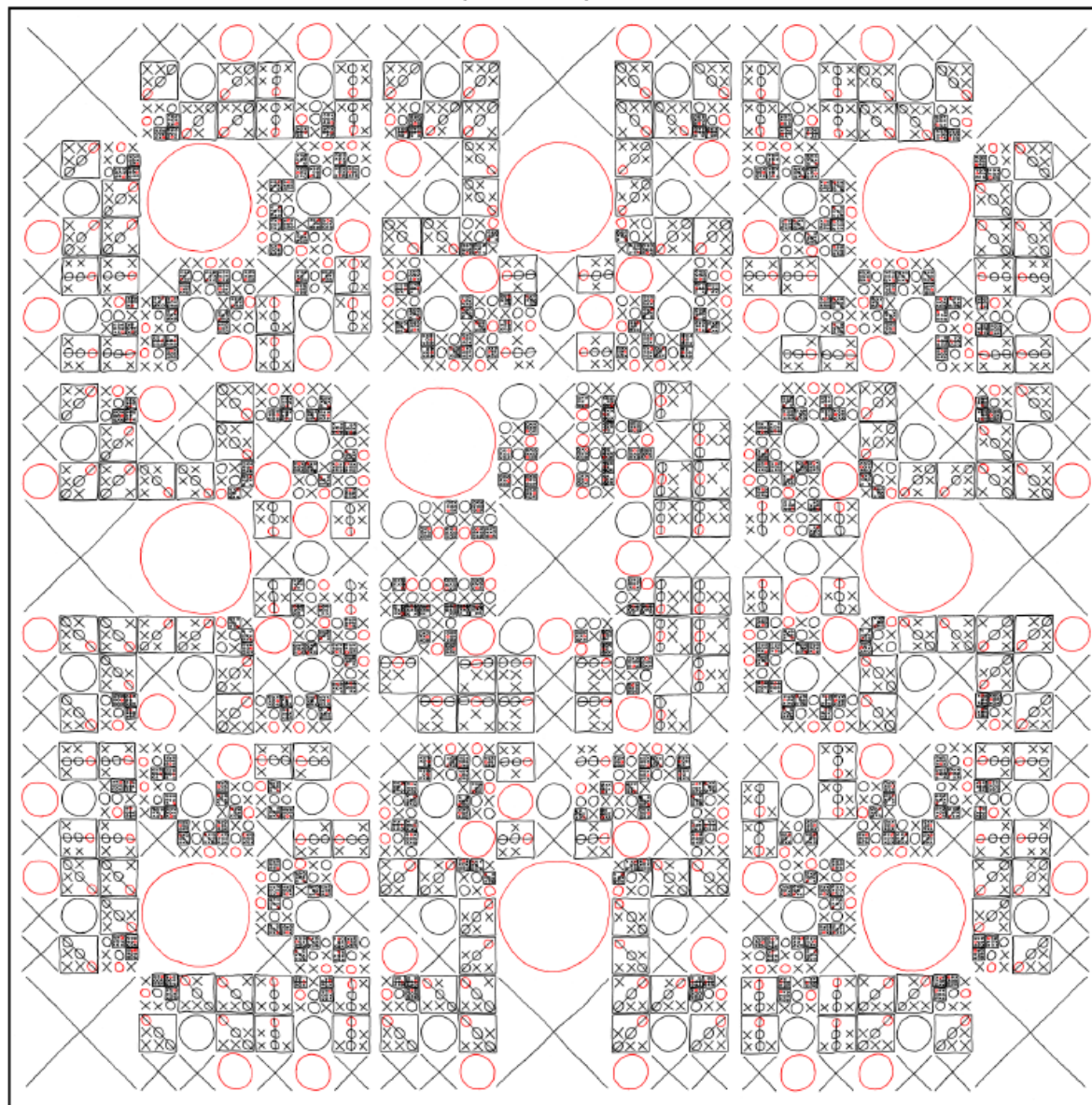
COMPLETE MAP OF OPTIMAL TIC-TAC-TOE MOVES

YOUR MOVE IS GIVEN BY THE POSITION OF THE LARGEST RED SYMBOL ON THE GRID. WHEN YOUR OPPONENT PICKS A MOVE, ZOOM IN ON THE REGION OF THE GRID WHERE THEY WENT. REPEAT.

MAP FOR X:

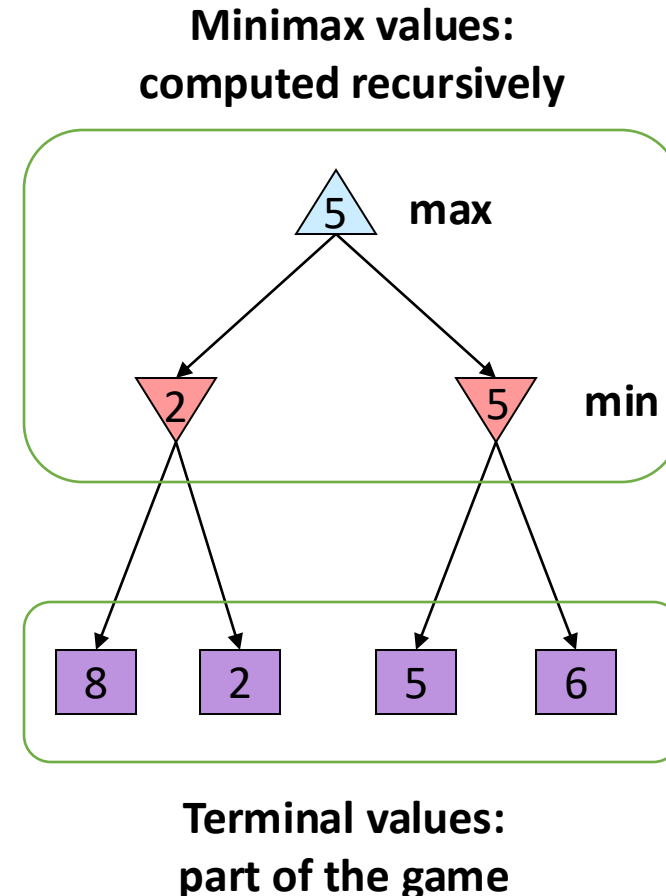


MAP FOR O:



Minimax Search: Adversarial Decision Making

- A state-space search tree representing all possible game states;
- Players alternate turns, choosing actions from their respective positions;
- At each node, compute the **minimax value**:
 - The best guaranteed utility assuming the opponent plays optimally and rationally.



Minimax Implementation

def value(state):

if the state is terminal: return the state's utility

if the next agent is MAX: return max-value(state)

if the next agent is MIN: return min-value(state)

def max-value(state):

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}))$

return v

def min-value(state):

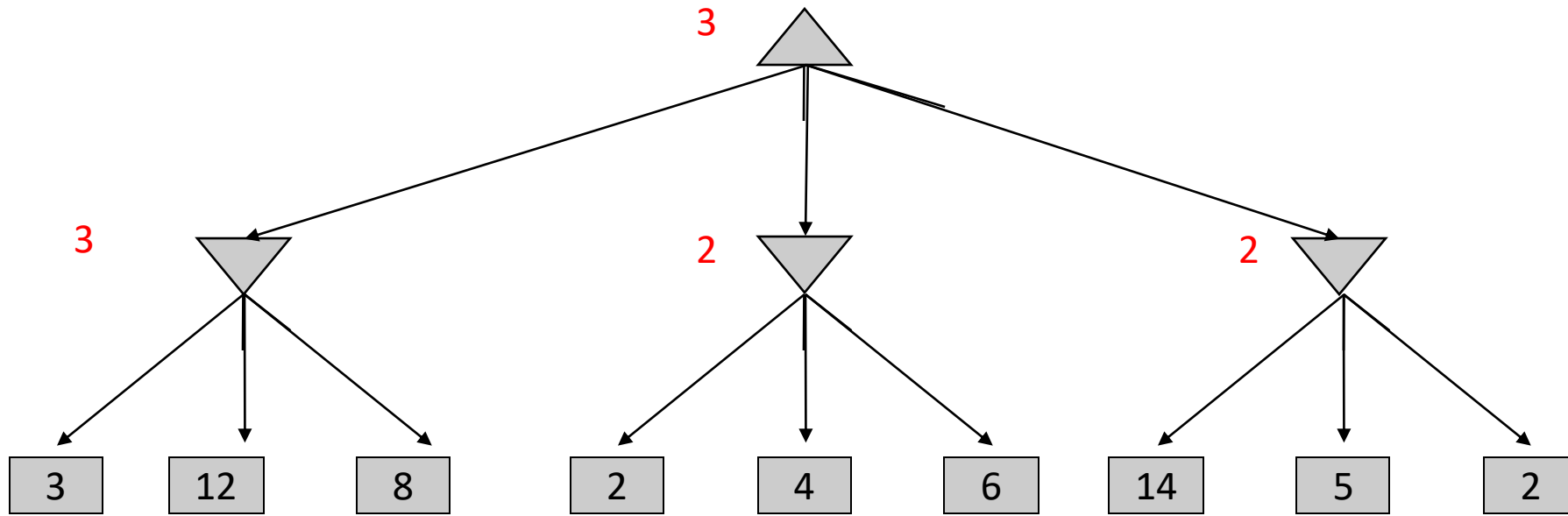
initialize $v = +\infty$

for each successor of state:

$v = \min(v, \text{value}(\text{successor}))$

return v

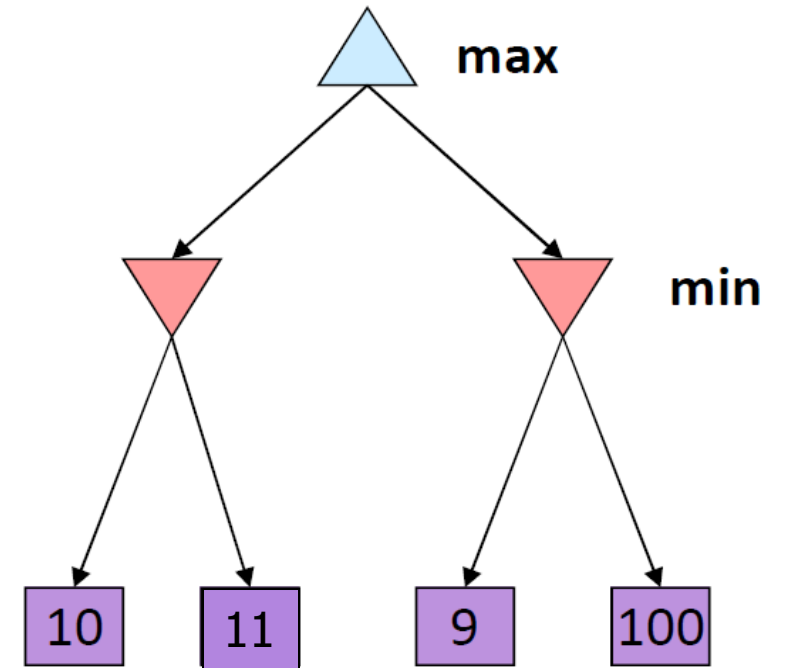
Minimax Example



- **Minimax value of a node:** The utility of a game state for the MAX player, assuming both players play optimally from that point onward.
- **Minimax strategy:** Select the move that yields the best possible outcome under the worst-case scenario.

Optimality of Minimax

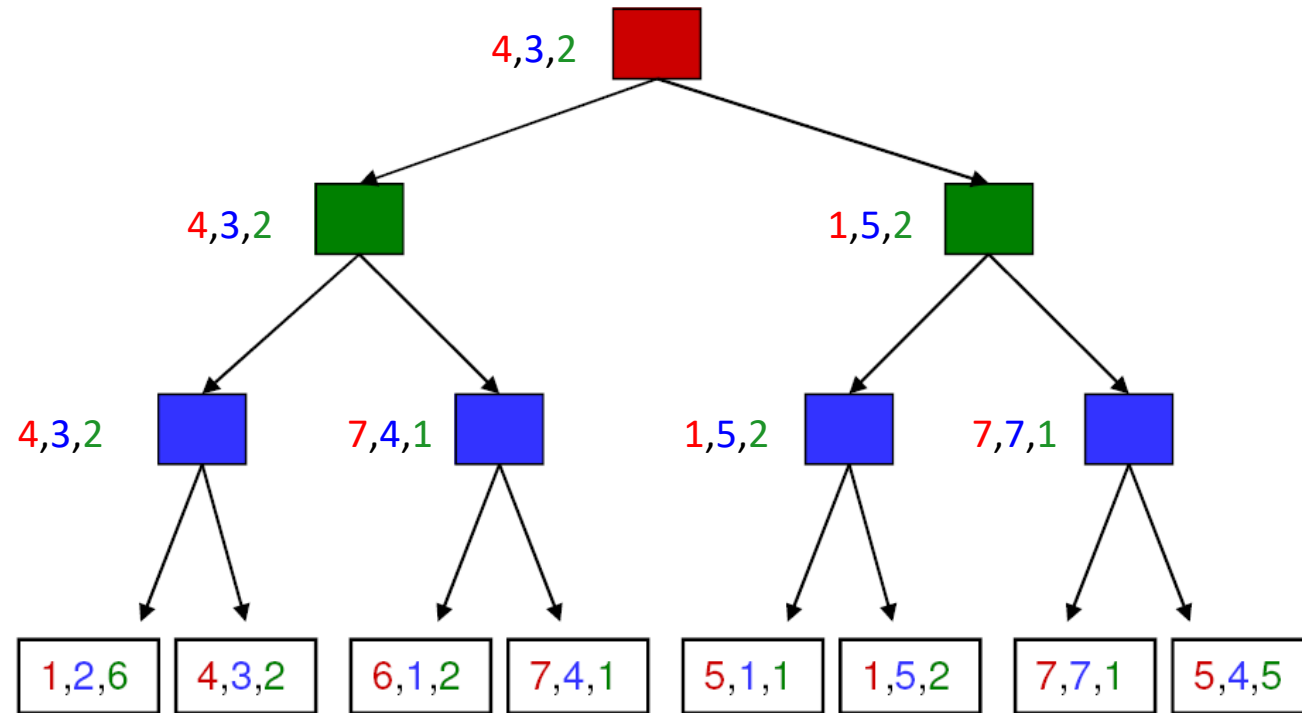
- The minimax search strategy is guaranteed optimal when facing an optimal (rational) opponent
- If the opponent is suboptimal, your actual utility may be higher than predicted by minimax
- However, while alternative strategies might exploit a suboptimal opponent more effectively,
 - They will perform worse against an optimal adversary;
 - Minimax remains the safest and most robust choice in uncertain or competitive settings



More Players, Non-zero-sum Case

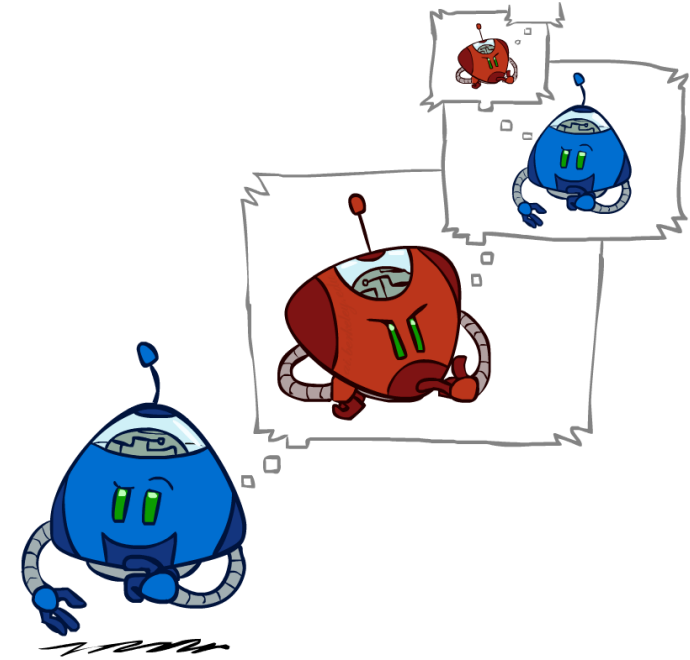
- Utilities are represented as tuples, with each element corresponding to a player's payoff.
- At each decision node, each player selects actions to maximize their own utility.
- Utility values are propagated upward from child nodes to parent nodes.
 - This allows strategic reasoning based on expected outcomes across all players.

More Players, Non-zero-sum Case

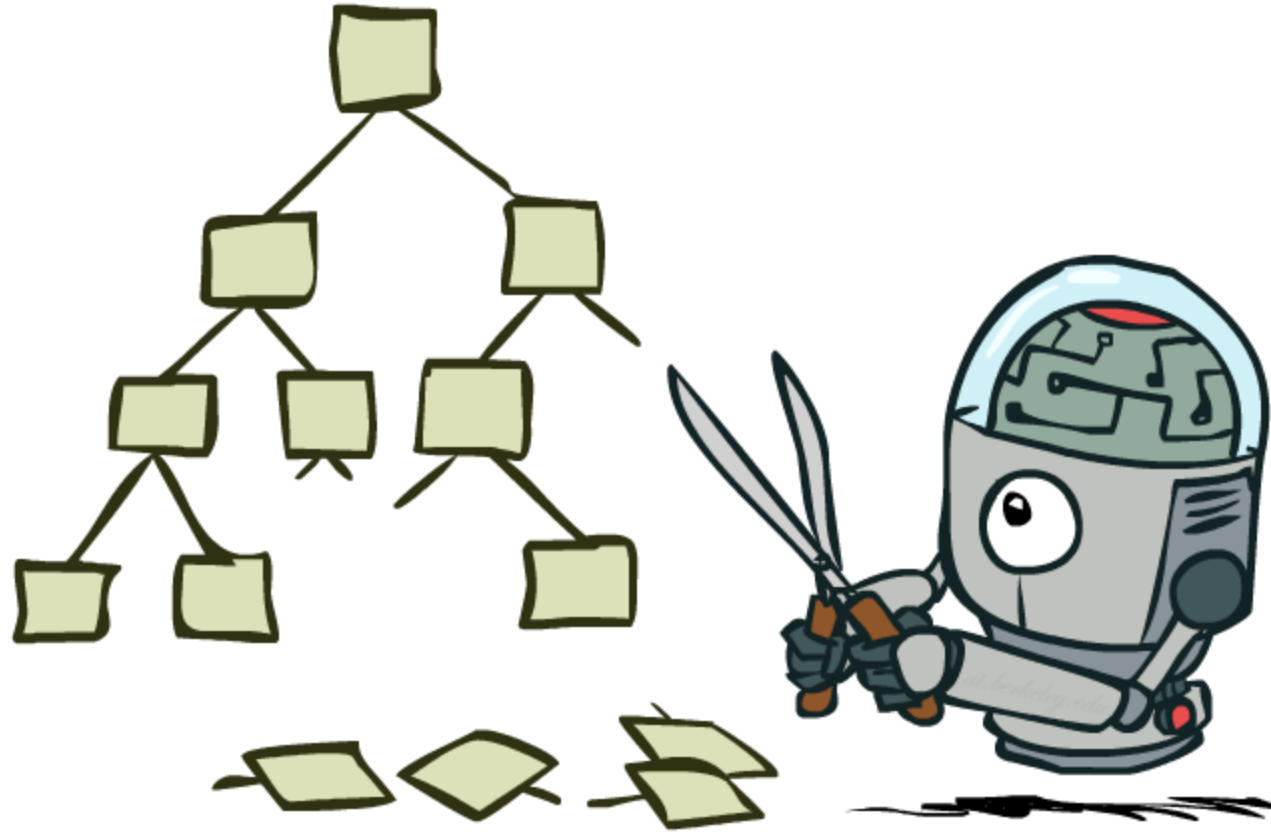


Efficiency of Minimax

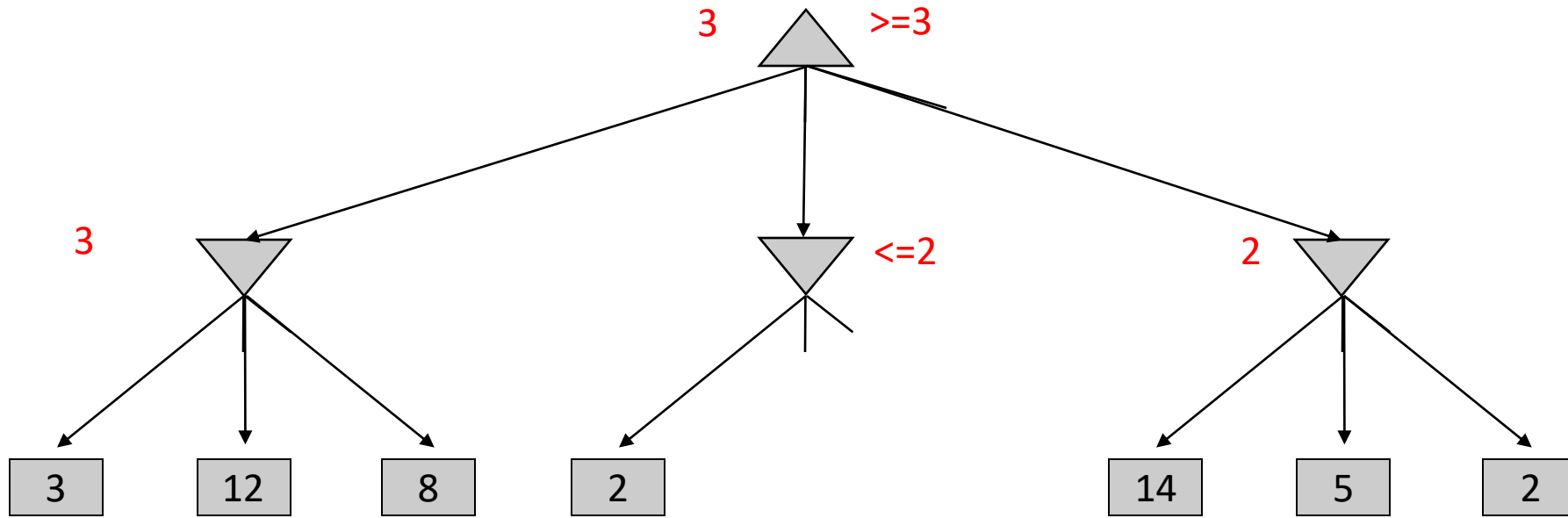
- How efficient is minimax?
 - Just like (exhaustive) DFS
 - Time: $O(b^m)$
 - Space: $O(bm)$
- Example: For chess, $b \approx 35$, $m \approx 100$
 - Exact solution is completely infeasible.
 - But, do we need to explore the whole tree?



Game Tree Pruning



Alpha-beta Pruning



It is possible to compute the exact minimax decision **without** expanding every node in the game tree.

Alpha-beta Pruning

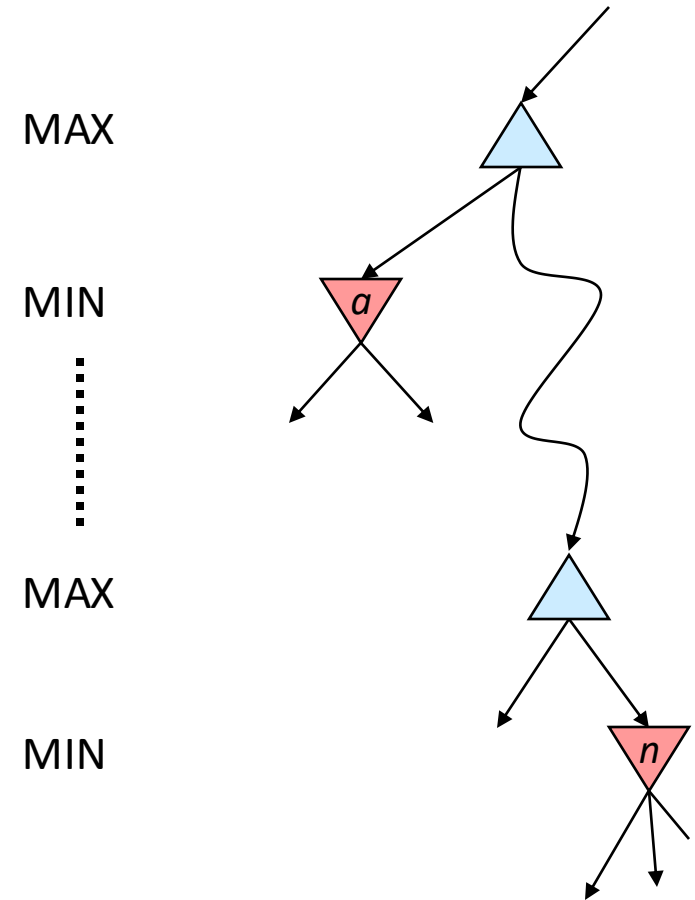
α : MAX's best option on path to root
 β : MIN's best option on path to root

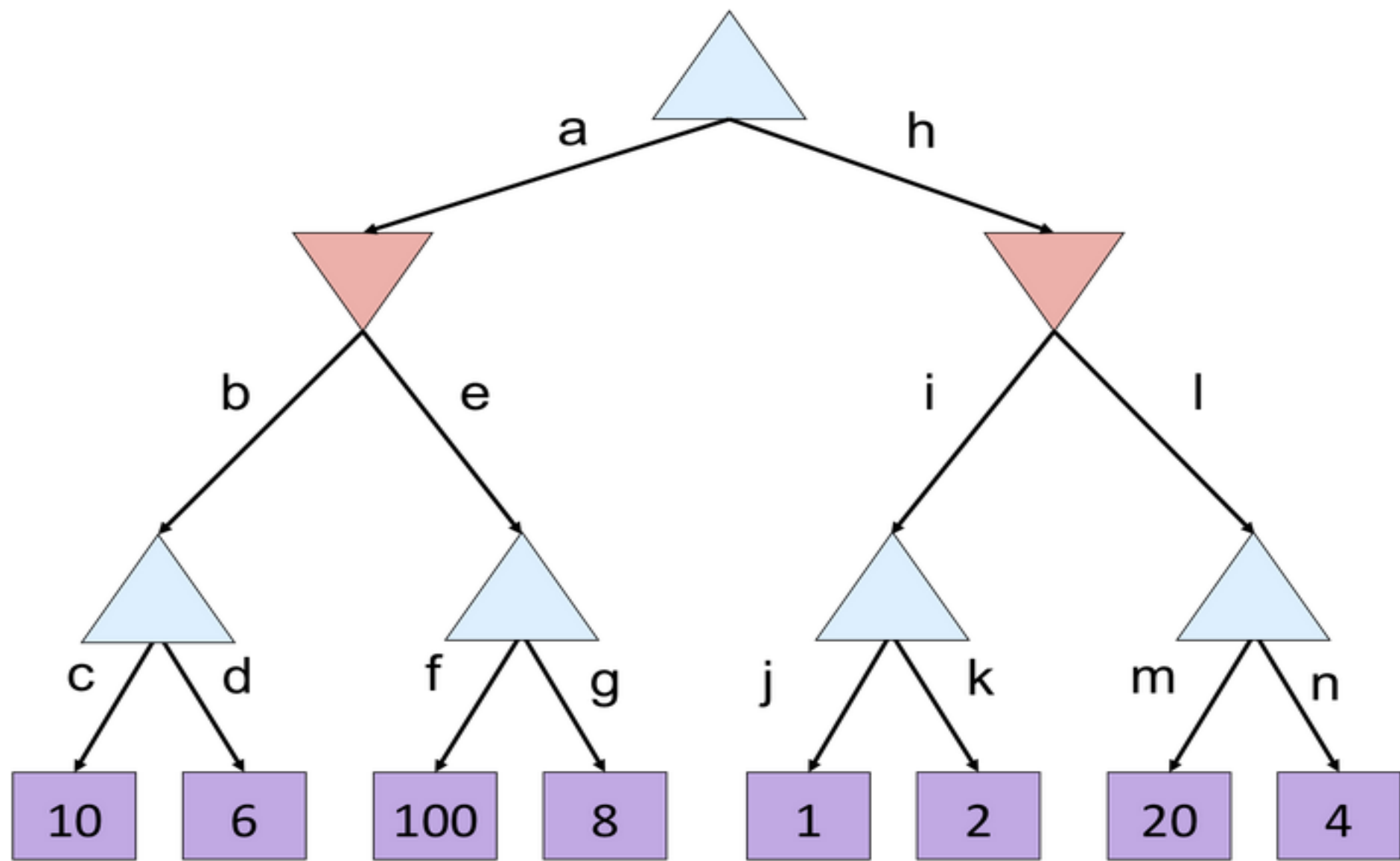
```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = -\infty$   
    for each successor of state:  
         $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \geq \beta$  return  $v$   
         $\alpha = \max(\alpha, v)$   
    return  $v$ 
```

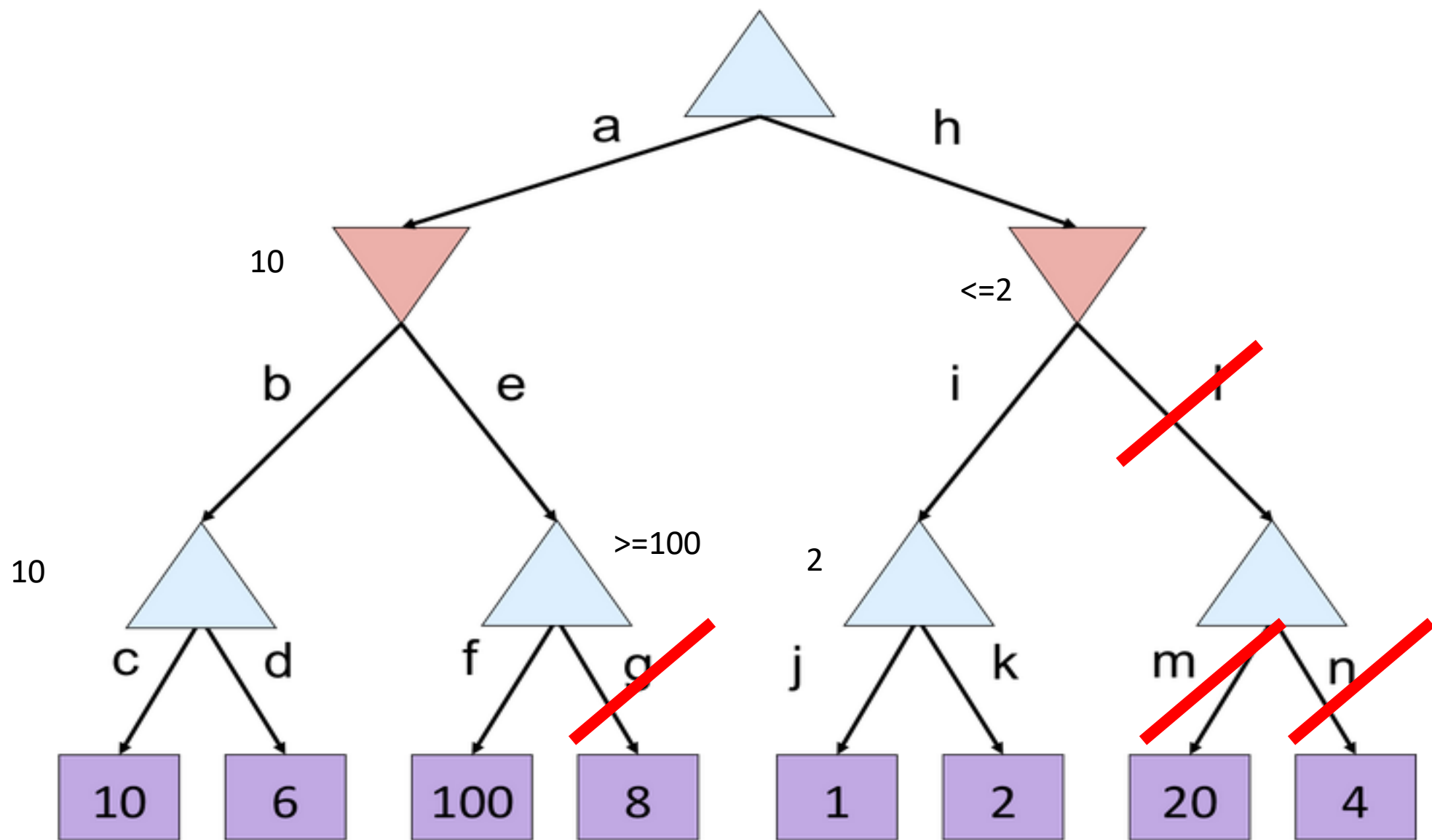
```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = +\infty$   
    for each successor of state:  
         $v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \leq \alpha$  return  $v$   
         $\beta = \min(\beta, v)$   
    return  $v$ 
```

Alpha-beta Pruning (MIN Version)

- We are computing the **MIN-VALUE** at a node n ;
- Looping through n 's children to evaluate possible outcomes;
- As we explore, n 's estimate of the minimum value **keeps decreasing**;
- Let α be the best value that **MAX** can guarantee along the current path from the root;
- If n 's value drops **below α** , MAX will **never choose this path**;
 - So we can **prune** the remaining children of n , as they won't affect the outcome.
- **MAX version is symmetric.**





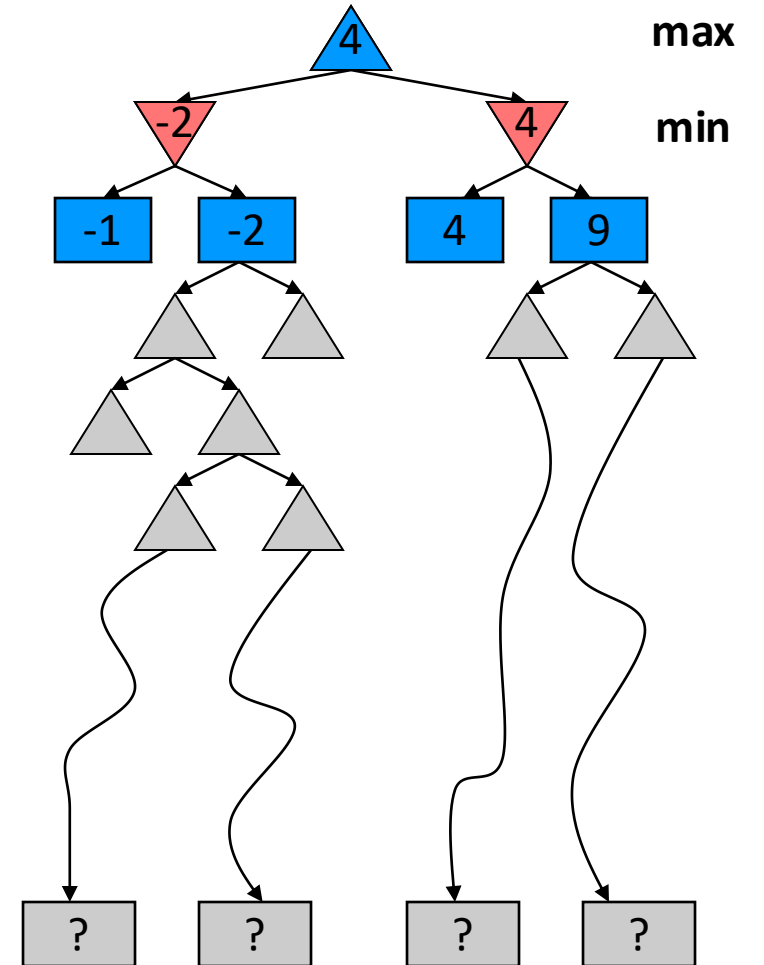


Resource Limits



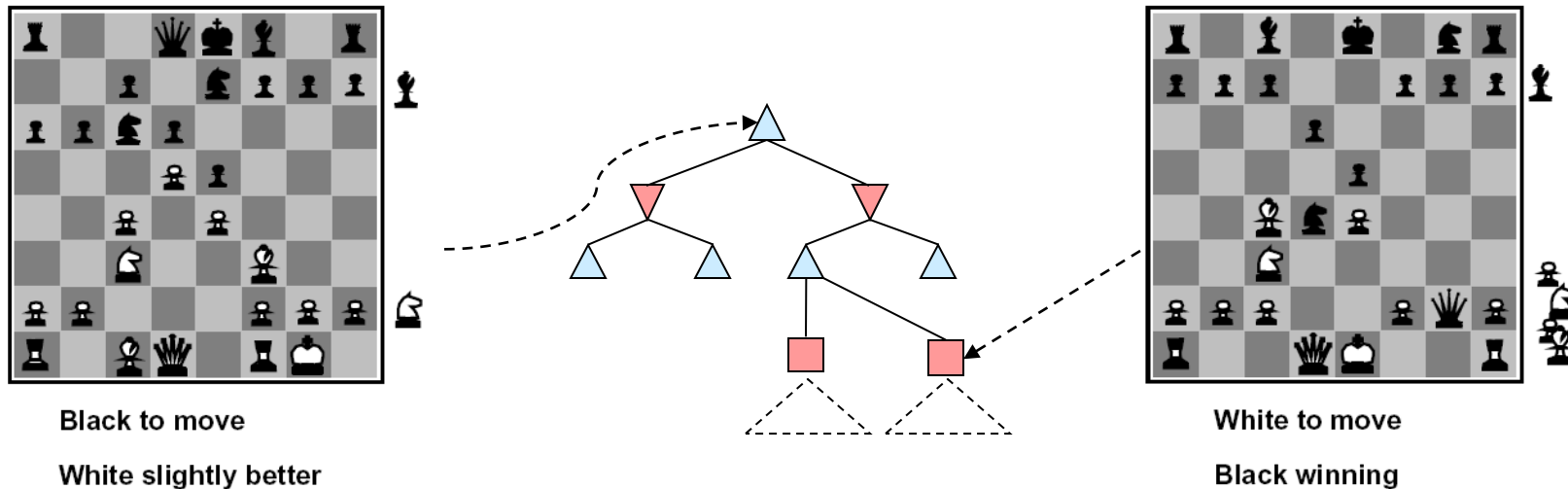
Resource Limits

- Problem: In realistic games (e.g., chess, Go), the search space is too large to reach terminal states within time constraints.
- Solution: **Depth-limited search**
 - Search only to a fixed depth in the game tree;
 - At non-terminal nodes, use an **evaluation function** to estimate utility;
 - This function approximates the desirability of a position without full rollout.



Evaluation Functions

- Evaluation functions estimated the scores for non-terminal states in depth-limited search.



Evaluation Function

- Ideal function: Returns the actual minimax value of the position.
- In practice: We approximate this using a weighted sum of features.

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- For example, in chess, w_k may be the material value of a piece (pawn = 1, knight = 3, rook = 5, queen = 9) and $f_k(s)$ may be the advantage in terms of that piece.

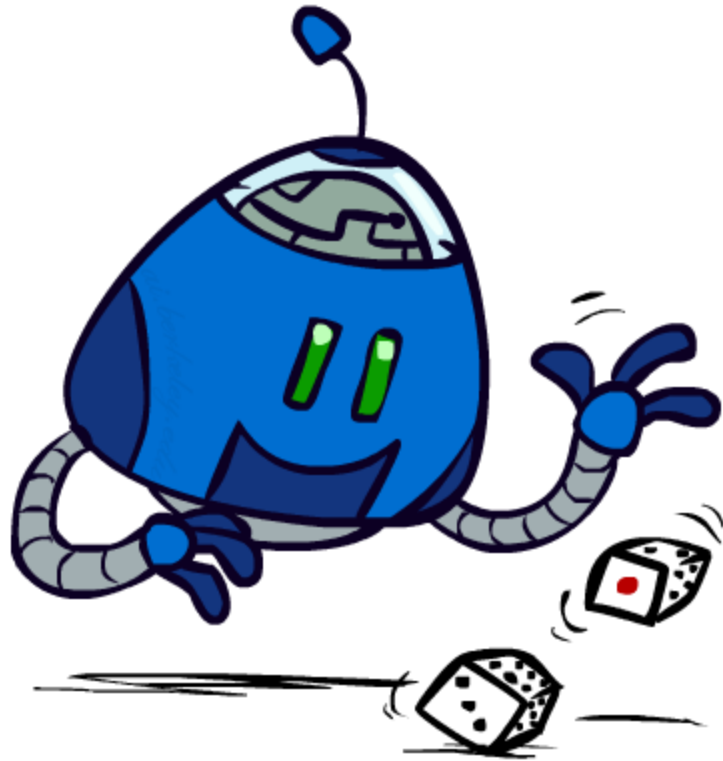
Evaluation Function

- Evaluation functions can be **learned from game databases**.
 - By analyzing large collections of expert-level games and outcomes
- Alternatively, they can be **trained through self-play**.
 - The program plays thousands or millions of games against itself.
 - Gradually improves its evaluation by reinforcing successful patterns.

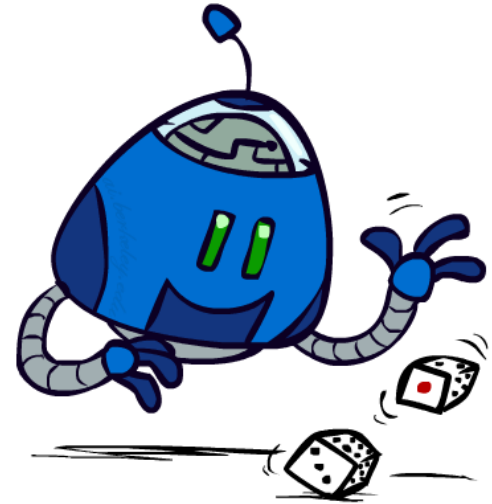
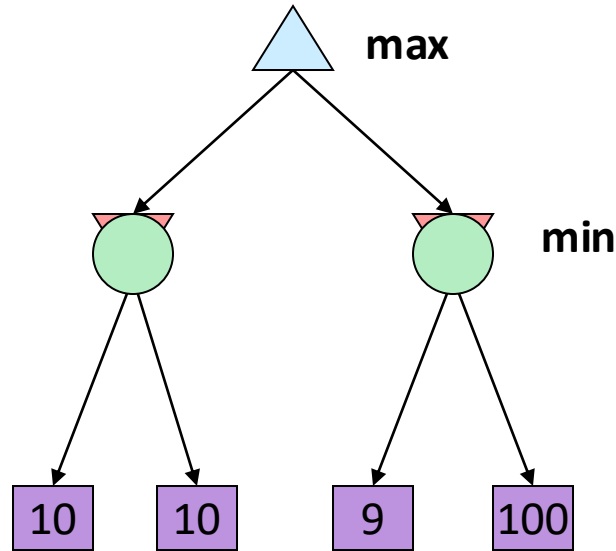
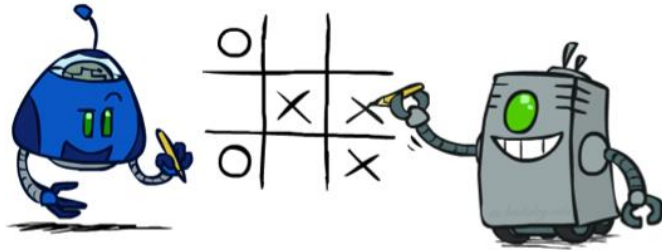
Depth Matters

- Evaluation functions are inherently imperfect
 - They can represent the probability of winning or the expected utility of a position.
- The deeper they are applied in the search tree, the less critical their precision becomes.
- This illustrates a key **tradeoff** between the **complexity of features** used in evaluation and the **computational cost** of applying them during search.

Uncertain Outcomes



Worst-Case vs. Average Case



Idea: Uncertain outcomes controlled by chance, not an adversary!

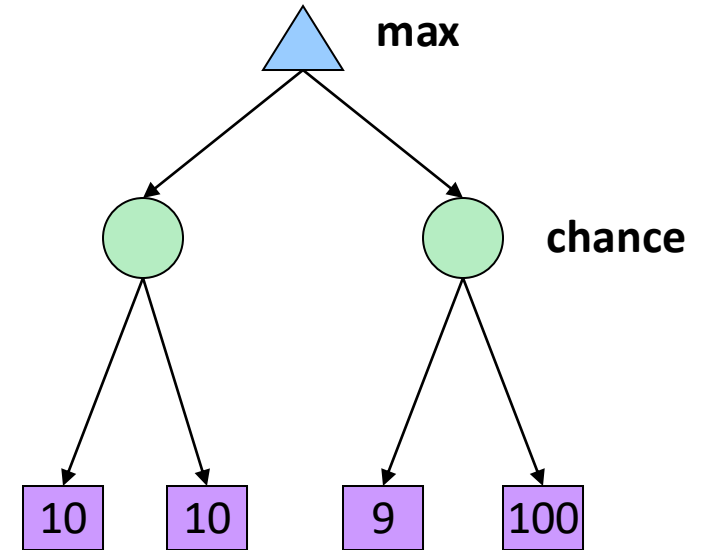
Why not Minimax?

- Worst case reasoning is too conservative.
- Need average case reasoning.



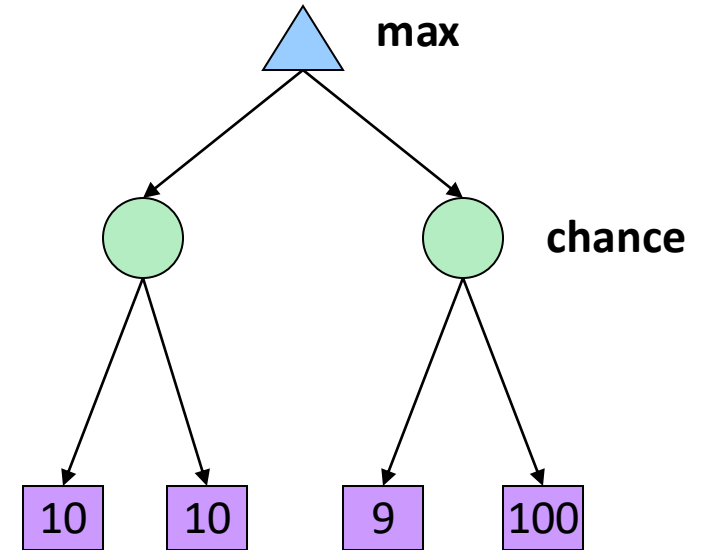
Expectimax Search

- Why wouldn't we know what the result of an action will be?
 - Explicit randomness: rolling dice
 - Unpredictable opponents: the ghosts respond randomly
 - Unpredictable humans: humans are not perfect
 - Actions can fail: when moving a robot, wheels might slip
- Values should now reflect average-case (expectimax) outcomes, not worst-case (minimax) outcomes.



Expectimax Search

- **Expectimax search**: compute the average score under optimal play.
 - **Max nodes** as in minimax search.
 - **Chance nodes** are like min nodes but the outcome is uncertain.
 - Calculate their **expected utilities**
 - i.e. take weighted average (expectation) of children.
- Later, we'll learn how to formalize the underlying uncertain-result problems as **Markov Decision Processes**.



Expectimax Pseudocode

```
def value(state):
```

```
    if the state is a terminal state: return the state's utility
```

```
    if the next agent is MAX: return max-value(state)
```

```
    if the next agent is EXP: return exp-value(state)
```

```
def max-value(state):
```

```
    initialize v =  $-\infty$ 
```

```
    for each successor of state:
```

```
        v = max(v, value(successor))
```

```
    return v
```

```
def exp-value(state):
```

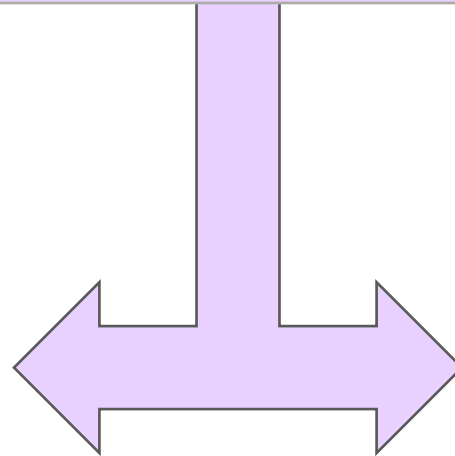
```
    initialize v = 0
```

```
    for each successor of state:
```

```
        p = probability(successor)
```

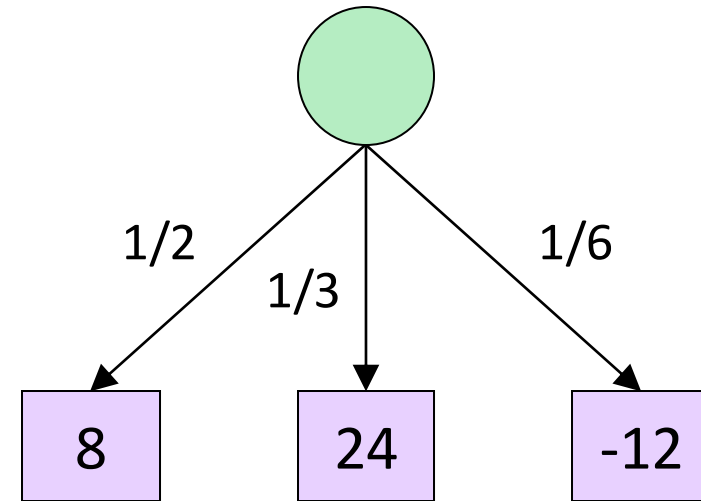
```
        v += p * value(successor)
```

```
    return v
```



Expectimax Pseudocode

```
def exp-value(state):  
    initialize v = 0  
    for each successor of state:  
        p = probability(successor)  
        v += p * value(successor)  
    return v
```



$$v = (1/2) (8) + (1/3) (24) + (1/6) (-12) = 10$$

Monte Carlo Tree Search

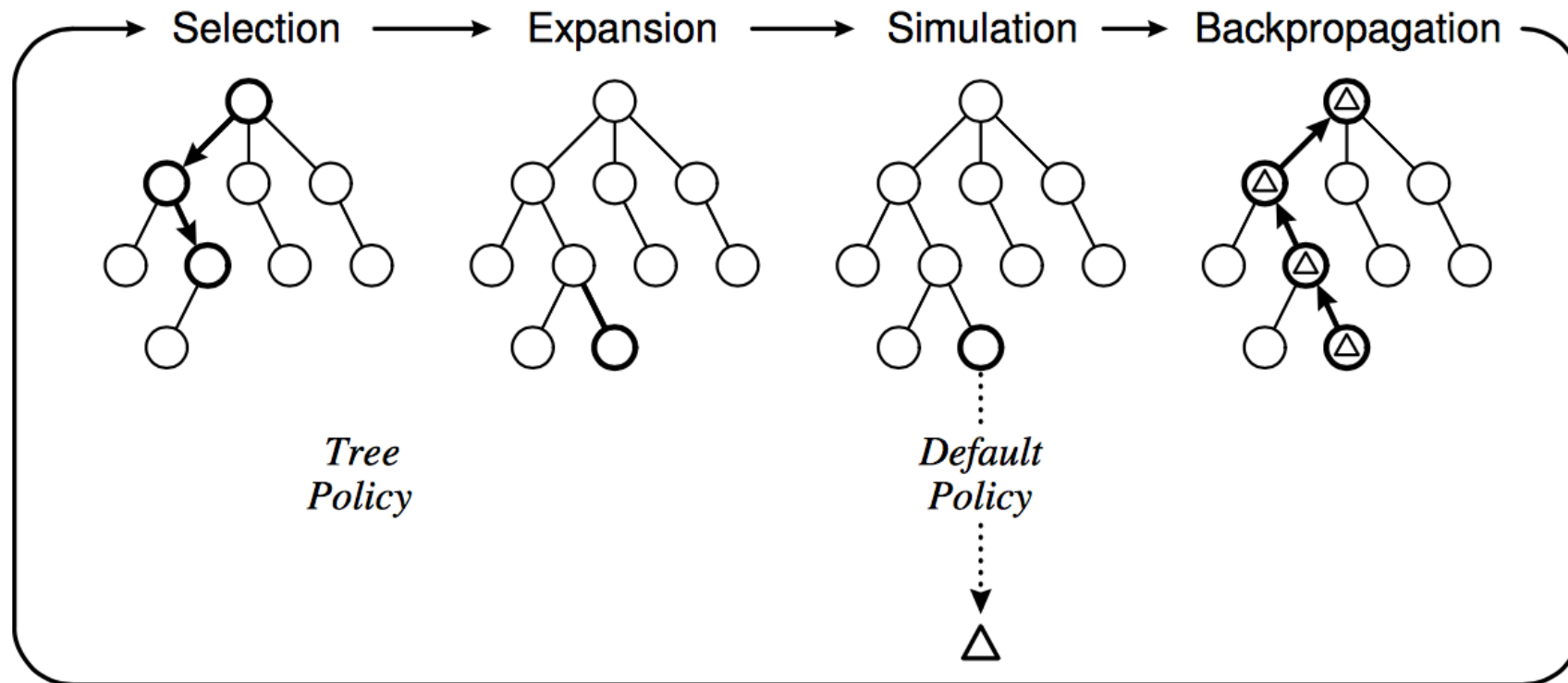
- Methods based on alpha-beta search assume a fixed cutoff depth.
 - Pretty hopeless for games like Go with $b > 300$.
- What about games with deep trees, large branching factor, and no good heuristics?
 - Instead of depth-limited search with an evaluation function, we use randomized simulations such as **Monte Carlo tree search (MCTS)**.

Monte Carlo Tree Search

- Unlike traditional search algorithms like minimax, MCTS doesn't rely on an explicit evaluation function or a complete search of the game tree.
- Instead, it uses random sampling and statistical analysis to guide the search.
 - **Evaluation by rollouts** – play multiple games to termination from a state s (using a simple, fast rollout policy) and count wins and losses.
 - **Selective search** – explore parts of the tree that will help improve the decision at the root, regardless of depth.

MCTS

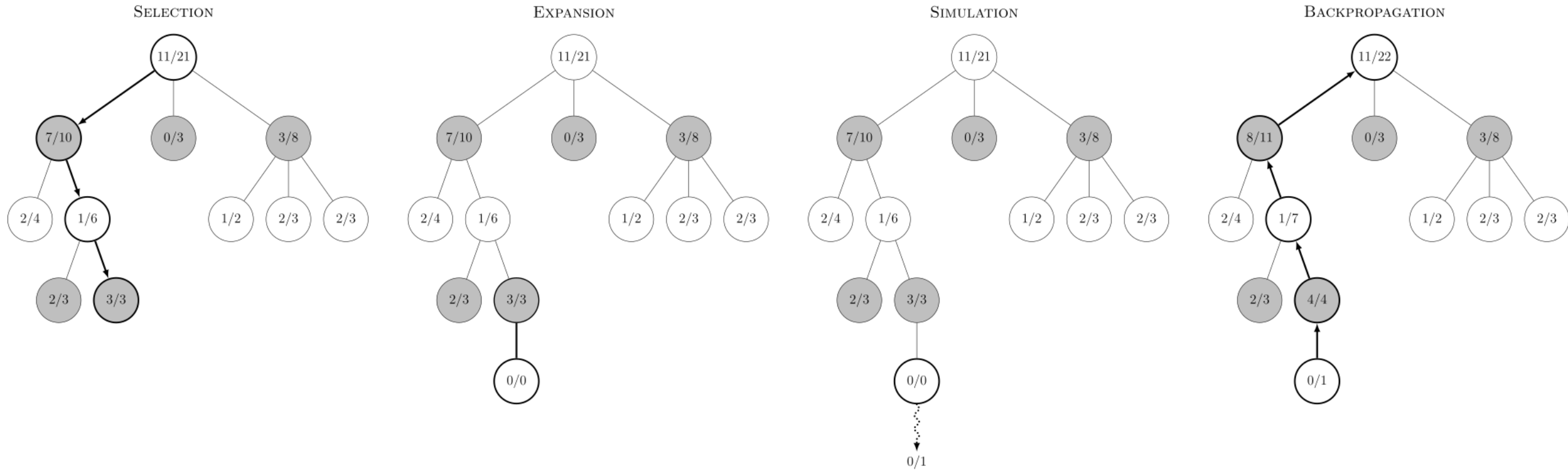
- MCTS starts with a single root node representing the current game state, and builds a search tree incrementally. .



MCTS

- **Selection:** MCTS employs a selection strategy to traverse the tree from the root node to a leaf node.
- **Expansion:** Once a leaf node is reached, MCTS expands it by generating child nodes representing possible moves from the current game state.
- **Simulation:** From each newly expanded node, MCTS conducts a simulation (also known as a **rollout**) by playing out the game randomly until a terminal state is reached. The outcome of the simulation is then used to evaluate the potential of the current move.
- **Backpropagation:** After the simulation, the results are backpropagated up the tree.

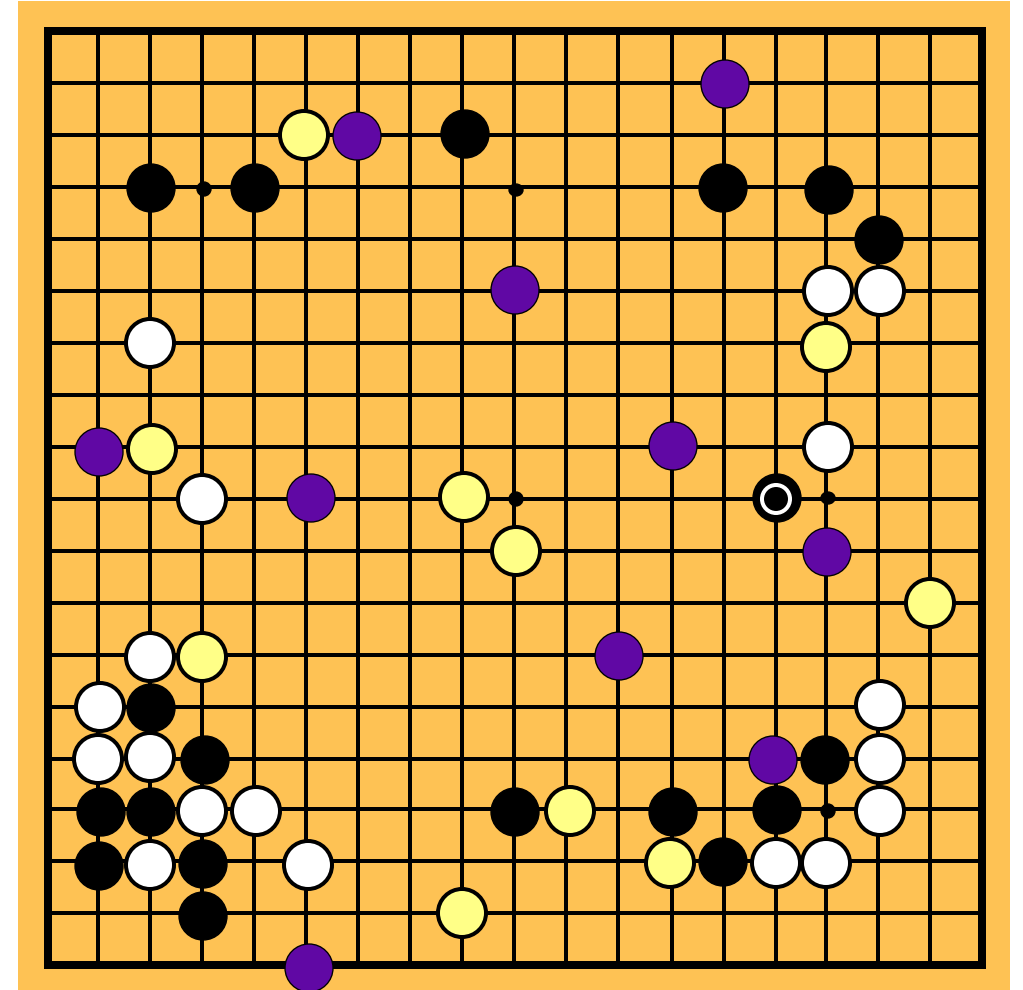
MCTS



Rollout

- For each rollout:
 - Repeat until terminal:
 - Play a move according to a fixed, fast rollout policy.
 - Record the result.
- Fraction of wins correlates with the true value of the position!
- Having a “better” rollout policy helps.

“Move 37”



AlphaGo with MCTS and Deep Learning

- In March 2016, it beat Lee Sedol in a five-game match, the first time a computer Go program has beaten a 9-dan professional.
- AlphaGo's algorithm uses a [Monte Carlo tree search](#) to find its moves based on knowledge previously "learned" by machine learning, specifically by an artificial neural network (a deep learning method) by extensive training, both from human and computer play.



Summary

- Games are important domain in artificial intelligence.
- Game theory has extended to many fields.
- Deep learning is fundamentally reshaping the world of games.

Thank You!